CSEN 296B-2: Week 1.2 Submission 2 — Initial Requirement Set

Student Name: Alex Shirazi
Date: April 1, 2026
Project: AstraNotes
Chosen Technical Path: Python

Architecture Context

Based on the architecture decision from Submission 1, AstraNotes uses a modular layered Python architecture with a repository interface pattern, file-based persistence, a privacy/encryption service, and a validation layer. The requirements below are written to align with and guide this architecture across the quarter.

Functional Requirements

ID	Requirement
FR-01	The system shall allow a user to create a new text note with a title and body.
FR-02	The system shall allow a user to edit an existing note and save the updated content, updating the last-modified timestamp automatically.
FR-03	The system shall allow a user to delete a note from the local collection by note identifier.
FR-04	The system shall allow a user to mark a note as private, which flags it for encryption handling by the privacy service.
FR-05	The system shall persist all notes to local file storage and restore them on application startup so that data survives between sessions.
FR-06	The system shall support note metadata including a unique identifier (UUID), created date, last modified date, and user-defined tags.
FR-07	The system shall allow a user to search notes by title or body content using keyword matching and return all matching results.
FR-08	The system shall display notes in a sorted list, ordered by last-modified date by default.

Non-Functional Requirements

ID	Requirement
NFR-01	The system shall handle a local collection of up to 500 notes without noticeable lag, completing note listing and keyword search within 2 seconds.
NFR-02	The system architecture shall separate core business logic from storage and presentation concerns so that each layer can be reviewed, tested, or replaced independently.

ID	Requirement
NFR-03	The application shall start and display the user's note list within 3 seconds on a machine meeting minimum development hardware specifications.

Security, Privacy, Reliability, and Governance Requirements

ID	Requirement
SPR-01	Private notes shall be encrypted at rest using Fernet symmetric encryption via Python's `cryptography` library so that raw note content is never stored in plaintext on disk.
SPR-02	The system shall catch all storage-related failures (file not found, permission denied, corrupt data) and surface clear, non-technical error messages instead of crashing or exposing stack traces.
SPR-03	All core components (repository, privacy service, validation layer) shall have at least one automated unit test verifying their primary behavior before each milestone delivery.
SPR-04	Any third-party library introduced into the project shall be documented with its purpose and license; unverified or unnecessary dependencies shall be avoided.

Traceability Summary

These 15 requirements (8 functional, 3 non-functional, 4 security/privacy/reliability/governance) are written to be directly traceable to:

- **Backlog items** — each requirement maps to one or more implementable user stories or tasks
- **Design decisions** — requirements like NFR-02 constrain the layered architecture established in Submission 1
- **Prototypes** — functional requirements FR-01 through FR-08 define what early prototypes must demonstrate
- **Tests** — each requirement includes a verifiable condition (e.g., "within 2 seconds," "encrypted at rest," "non-technical error messages") that can become a test case
- **Acceptance criteria** — the specificity of each requirement makes it possible to determine pass or fail at review time

---

Short Reflection

Generating requirements with AI was fast, but the real work was in refining them. The first AI-generated set was too generic — it listed things like "the system should save notes" without specifying format, metadata, or failure behavior. I also had to push back on overlapping requirements: the AI initially listed separate items for "persist to storage" and "load on startup" that were really two sides of the same durability concern. By iterating on the prompt and critically reviewing the output for vagueness and redundancy, I arrived at a set that I can actually use to drive backlog creation and architecture validation across the quarter.